

GoodWe ChargeGrid Intelligence

Sistema Integrado de Gestão de Eletropostos

Sprint 1 - 2026

1CCR - EQUIPE 04

INTEGRANTES:

- Gabriela Angel Silva — RM: 570808
- Izabelly Menezes — RM: 570673
- Marcos Paulo Sampaio — RM: 573987
- Otávio Santos — RM: 570225
- Tiago Muhlmann — RM: 569569
- Wesley Marques — RM: 573915

CURSO: Ciência da Computação - 1º Ano

INSTITUIÇÃO: FIAP

2. IMPLEMENTAÇÃO EM LINGUAGEM C

2.1. Visão Geral do Programa

O programa desenvolvido na Sprint 1 do projeto ChargeGrid Intelligence é um sistema de gestão de sessões de recarga de veículos elétricos implementado em linguagem C.

A execução ocorre via interface de menu interativo em terminal, sem dependências externas além das bibliotecas padrão da linguagem.

O objetivo do programa é simular o funcionamento de um eletroposto, permitindo ao operador: registrar os dados do motorista e do veículo, configurar uma sessão de recarga com cálculo automático de energia e custo, e visualizar o relatório final de cobrança.

O fluxo é controlado por um loop principal que permanece ativo até o operador encerrar a aplicação.

As principais características do programa são:

- Linguagem: C
- Compilador: GCC
- Bibliotecas utilizadas: stdio.h e string.h
- Interface: terminal (linha de comando)
- Paradigma: procedural / imperativo
- Persistência: nenhuma — estado mantido em variáveis globais durante a execução

2.2. Estrutura do Código

2.2.1. Bibliotecas Utilizadas

O programa utiliza exclusivamente bibliotecas da biblioteca padrão C (libc), garantindo portabilidade total entre plataformas:

```
#include <stdio.h>
#include <string.h>
```

stdio.h — fornece as funções de entrada e saída utilizadas no programa: **printf()**, **scanf()** e **getchar()**.

string.h — fornece a função **strlen()**, utilizada exclusivamente na validação do comprimento da placa do veículo.

2.2.2. Definições e Constantes

Três constantes simbólicas são definidas com a diretiva **#define**, separando os parâmetros de negócio do restante do código lógico. Isso facilita futuras alterações de tarifação sem necessidade de modificar o interior das funções:

```
#define TAXA_VISITANTE 5.00f
#define TAXA_PREMIUM 5.00f
#define DESC_PREMIUM 0.85f
```

TAXA_VISITANTE — taxa de serviço de R\$ 5,00 cobrada por sessão para usuários do tipo Visitante.

TAXA_PREMIUM — taxa de serviço de R\$ 5,00 cobrada por sessão para usuários do tipo Premium.

DESC_PREMIUM — fator multiplicador de 0,85, que representa o desconto de 15% aplicado sobre o custo de energia do cliente Premium.

2.2.3. Estruturas de Dados

Devido ao escopo desta Sprint 1, o programa não utiliza structs ou tipos compostos. Todos os dados são mantidos em variáveis globais, o que simplifica o compartilhamento de estado entre as funções sem necessidade de passagem de parâmetros:

```
/* Dados do usuário */
char nome[50];
char placa[8];
int tipo_usuario;

/* Dados da sessão */
int hr_inicio;
int duracao_min;
float potencia_kw;
float energia_kwh;

/* Tarifação */
float tarifa;
float custo_total;

/* Controle de fluxo */
int sessao_iniciada = 0;
int sessao_configurada = 0;
```

nome[50] — armazena o nome do motorista, aceitando até 49 caracteres mais o terminador nulo.

placa[8] — armazena a placa do veículo. A validação exige exatamente 7 caracteres.

tipo_usuario — inteiro que representa o perfil: 1 = Visitante, 2 = Mensalista, 3 = Premium.

hr_inicio — hora de início da sessão (0 a 23), usada para determinar a faixa tarifária.

duracao_min — duração da sessão em minutos, base para o cálculo da energia consumida.

potencia_kw — potência do modo de recarga selecionado, em kW.

energia_kwh — energia total consumida calculada, em kWh.

tarifa — valor da tarifa aplicada em R\$/kWh, conforme o horário de início.

custo_total — custo final da sessão em reais, após aplicação das regras de perfil.

sessao_iniciada e **sessao_configurada** — flags binárias que controlam o fluxo do menu, impedindo que o usuário avance para etapas sem completar as anteriores.

2.3. Funções Principais

O programa é composto por quatro funções além de main(). Cada uma encapsula uma responsabilidade bem definida, seguindo o princípio de separação de responsabilidades:

2.3.1. Início de Sessão — `inserir_usuario()`

Esta função é o ponto de entrada do fluxo. Ela coleta os dados de identificação do motorista e valida cada campo antes de prosseguir, utilizando laços do-while para repetir a leitura em caso de entrada inválida.

Etapas executadas pela função:

- Leitura do nome com o especificador `%49[^\n]`, que aceita espaços e limita a 49 caracteres.
- Leitura e validação da placa: o laço do-while repete até que `strlen(placa)` seja igual a 7. Após cada leitura, `limpar_buffer()` é chamada para evitar resíduos no stdin.
- Seleção do tipo de usuário (1 a 3) com validação de intervalo dentro de um laço do-while.
- Ao final, `sessao_iniciada` recebe o valor 1 e `sessao_configurada` é redefinida como 0, garantindo que uma nova sessão precise ser configurada.

```
void inserir_usuario() {
    printf("\nNome do motorista: ");
    scanf(" %49[^\n]", nome);

    do {
        printf("Placa do veiculo (7 caracteres): ");
        scanf(" %49s", placa);
        limpar_buffer();
        if (strlen(placa) != 7)
            printf("\nPlaca invalida! Tente novamente.\n");
    } while (strlen(placa) != 7);

    /* ... seleção do tipo_usuario via do-while ... */

    sessao_iniciada = 1;
    sessao_configurada = 0;
    printf("Dados inseridos com sucesso!\n");
}
```

2.3.2. Simulação de Recarga — `inserir_sessao()`

Responsável por coletar os parâmetros técnicos da sessão de recarga. Antes de prosseguir, verifica se o usuário foi cadastrado por meio da flag `sessao_iniciada`. Calcula automaticamente a energia consumida e a tarifa aplicável conforme o horário informado.

Parâmetros coletados:

- Hora de início: inteiro de 0 a 23, validado com do-while.
- Duração: inteiro maior que zero, representando os minutos da sessão.
- Modo de recarga: seleção entre três opções de potência processadas por switch — 7,4 kW (lento), 11,0 kW (médio) ou 22,0 kW (rápido).

Cálculos realizados internamente:

- Energia consumida: $\text{energia_kwh} = \text{potencia_kw} \times (\text{duracao_min} / 60.0f)$
- Tarifa horária: R\$ 1,85/kWh no período de ponta (18h–21h); R\$ 0,85/kWh fora de ponta (22h–6h); R\$ 1,20/kWh no período normal.
- Custo base: $\text{custo_total} = \text{energia_kwh} \times \text{tarifa}$
- Ajuste por perfil: Visitante soma R\$ 5,00; Premium aplica fator 0,85 e soma R\$ 5,00; Mensalista não recebe ajuste.

```
/* Seleção da potência */
switch (modo) {
    case 1: potencia_kw = 7.4f; break;
    case 2: potencia_kw = 11.0f; break;
    case 3: potencia_kw = 22.0f; break;
    default: printf("Modo invalido! Tente novamente.\n");
}

/* Cálculo de energia e tarifa */
float horas = duracao_min / 60.0f;
energia_kwh = potencia_kw * horas;

if      (hr_inicio >= 18 && hr_inicio < 21)  tarifa = 1.85f;
else if (hr_inicio >= 22 || hr_inicio < 6)   tarifa = 0.85f;
else                                         tarifa = 1.20f;

custo_total = energia_kwh * tarifa;

/* Ajuste por perfil de usuário */
if      (tipo_usuario == 1) custo_total += TAXA_VISITANTE;
else if (tipo_usuario == 3) custo_total = (custo_total * DESC_PREMIUM) +
TAXA_PREMIUM;
```

2.3.3. Cálculo de Tarifação

A tarifação é determinada exclusivamente pela hora de início informada pelo operador. O sistema reconhece três faixas horárias:

- Período de Ponta (18h00 às 20h59): tarifa de R\$ 1,85/kWh, correspondente ao horário de maior demanda da rede elétrica.
- Período Normal (06h00 às 17h59): tarifa de R\$ 1,20/kWh, aplicada na maior parte do dia.
- Fora de Ponta (22h00 às 05h59): tarifa de R\$ 0,85/kWh, incentivando o uso noturno do eletroposto.

Após o cálculo do custo base (energia × tarifa), são aplicados os ajustes conforme o perfil do usuário:

- Visitante: $\text{custo_total} = \text{custo_base} + \text{R\$ } 5,00$
- Mensalista: $\text{custo_total} = \text{custo_base}$ (sem ajuste)
- Premium: $\text{custo_total} = (\text{custo_base} \times 0,85) + \text{R\$ } 5,00$

2.3.4. Relatório de Sessão — `exibir_resultado()`

Formata e imprime no terminal um relatório estruturado com todos os dados da sessão. Antes de exibir qualquer informação, a função verifica as duas flags de estado; caso alguma esteja inativa, exibe uma mensagem orientativa e retorna imediatamente.

O relatório exibe: nome e placa do motorista, perfil do usuário, hora de início, duração, potência e energia da sessão, período tarifário, tarifa aplicada, desconto ou isenção conforme o perfil, e o valor total a pagar.

```
void exibir_resultado() {
    if (!sessao_iniciada) { printf("Nenhuma sessao iniciada!\n"); return; }
    if (!sessao_configurada) { printf("Configure a sessao (opcao 2).\n"); return; }

    const char *perfil = (tipo_usuario == 1) ? "Visitante" :
                          (tipo_usuario == 2) ? "Mensalista" : "Premium";

    const char *periodo = (hr_inicio >= 18 && hr_inicio < 21) ? "PONTA" :
                          (hr_inicio >= 22 || hr_inicio < 6) ? "FORA DE
PONTA" : "NORMAL";

    printf("\n==== RESULTADO DA SESSAO =====\n");
    printf("Motorista : %s\n", nome);
    printf("Placa      : %s\n", placa);
    printf("Perfil      : %s\n", perfil);
    /* ... demais campos ... */
    printf("TOTAL       : R$ %.2f\n", custo_total);
    printf("===== \n");
}
```

2.4. Estruturas de Controle

2.4.1. Condicionais (if/else/switch)

O programa emprega estruturas condicionais em três contextos distintos:

Determinação da tarifa horária: cadeia if-else if-else que mapeia a hora de início da sessão para a faixa tarifária correspondente, conforme as regras descritas na seção 6.3.3.

```
if      (hr_inicio >= 18 && hr_inicio < 21)  tarifa = 1.85f;
else if (hr_inicio >= 22 || hr_inicio < 6)   tarifa = 0.85f;
else                                         tarifa = 1.20f;
```

Ajuste por perfil de usuário: condicional if-else if aplicada após o cálculo do custo base. O Mensalista (tipo 2) é tratado implicitamente pela ausência de qualquer ajuste.

```
if      (tipo_usuario == 1)  custo_total += TAXA_VISITANTE;
else if (tipo_usuario == 3)  custo_total = (custo_total * DESC_PREMIUM) +
TAXA_PREMIUM;
```

Seleção de potência via switch: o switch processa a opção do modo de recarga. O caso default trata entradas inválidas sem encerrar o programa, mantendo o laço de seleção ativo.

```
switch (modo) {
    case 1: potencia_kw = 7.4f; break;
    case 2: potencia_kw = 11.0f; break;
    case 3: potencia_kw = 22.0f; break;
    default: printf("Modo invalido! Tente novamente.\n");
}
```

2.4.2. Repetição (for/while/do-while)

Todas as validações de entrada utilizam laços do-while, garantindo que pelo menos uma leitura ocorra antes da verificação de validade. O loop principal do programa utiliza while:

Loop principal — while: permanece ativo até o usuário selecionar a opção 4 (sair). Trata entradas não-numéricas chamando `limpar_buffer()` e reiniciando o ciclo, evitando loop infinito por falha no `scanf`.

```
while (opcao != 4) {
    printf("\n1 - INSERIR INFORMACOES DO USUARIO\n");
    printf("2 - INSERIR INFORMACOES DA SESSAO\n");
    printf("3 - EXIBIR RESULTADO\n");
    printf("4 - SAIR\n");
    printf("\nDigite a opcao desejada: ");

    if (scanf("%d", &opcao) != 1) {
        limpar_buffer();
        opcao = 0;
        printf("OPCAO INVALIDA\n");
        continue;
    }
    switch (opcao) { /* ... */ }
}
```

Validação de campos — do-while: padrão utilizado para todos os campos numéricos (tipo de usuário, hora de início, duração e modo de recarga) e para a placa do veículo. O laço garante que a variável receba um valor válido antes de prosseguir.

```
do {
    printf("Placa do veiculo (7 caracteres): ");
    scanf(" %49s", placa);
    limpar_buffer();
    if (strlen(placa) != 7)
        printf("\nPlaca invalida! Tente novamente.\n");
} while (strlen(placa) != 7);
```

2.5. Entrada e Saída de Dados

Toda interação com o usuário é realizada via terminal por meio de `printf()` para saída e `scanf()` para entrada, ambas pertencentes à biblioteca `stdio.h`.

Leitura de dados — formatos utilizados no `scanf`:

- `%49[^\n]` — leitura do nome, aceita espaços; o modificador `^\n` impede que a leitura pare no primeiro espaço.
- `%49s` — leitura da placa, para no primeiro espaço; o comprimento é validado com `strlen()`.
- `%d` — leitura de todos os campos inteiros (tipo de usuário, hora de início, duração e modo de recarga); o valor de retorno do `scanf` é verificado para detectar entradas não-numéricas.

Proteção do buffer — `limpar_buffer()`: função utilitária chamada após toda leitura de string ou após detecção de falha no `scanf`. Descarta todos os caracteres pendentes no `stdin` até o próximo `\n`. Sem ela, um `scanf` de inteiro que falha deixa o caractere inválido no buffer, causando loop infinito na próxima chamada.

```
void limpar_buffer() {
    while (getchar() != '\n')
        continue;
}
```

Saída formatada — especificadores utilizados no printf:

- %s — exibe strings (nome, placa, perfil e período tarifário).
- %d — exibe inteiros (hora de início e duração em minutos).
- %.1f — exibe a potência com uma casa decimal (ex.: 22.0 kW).
- %.2f — exibe valores monetários e energia com duas casas decimais (ex.: R\$ 28.84, 33.00 kWh).
- %02d — exibe a hora com zero à esquerda quando necessário (ex.: 09h).

2.6. Exemplo de Execução

A seguir é apresentada uma execução completa simulando um cliente Premium realizando uma recarga rápida no período de fora de ponta (23h).

Dados inseridos pelo operador:

```
Nome do motorista: Izabelly Menezes
Placa do veículo (7 caracteres): CCR1TDS

Tipos de usuario
1 - Visitante
2 - Mensalista
3 - Premium
Digite: 3
Dados inseridos com sucesso!
```

```
Digite a opcao desejada: 2
Hora de inicio (0-23): 23
Duracao da recarga em minutos: 90
Modo de recarga:
1 - 7.4 kW (Lento)
2 - 11.0 kW (Medio)
3 - 22.0 kW (Rapido)
Digite: 3
Sessao configurada e calculada com sucesso.
```

```
1 - INSERIR INFORMACOES DO USUARIO
2 - INSERIR INFORMACOES DA SESSAO
3 - EXIBIR RESULTADO
4 - SAIR
```

```
Digite a opcao desejada: 4
FINALIZANDO...
```


Saída exibida pelo programa no terminal:

```
Digite a opcao desejada: 3

===== RESULTADO DA SESSAO =====
Motorista : Izabelly Menezes
Placa     : CCR1TDS
Perfil    : Premium

--- Sessao ---
Hora inicio: 23h
Duracao    : 90 minutos
Potencia   : 22.0 kW
Energia    : 33.00 kWh

--- Cobranca ---
Periodo    : FORA DE PONTA
Tarifa     : R$ 0.85/kWh
Desconto   : 15% (sobre energia)
Taxa servico: R$ 5.00
TOTAL      : R$ 28.84
=====

1 - INSERIR INFORMACOES DO USUARIO
2 - INSERIR INFORMACOES DA SESSAO
3 - EXIBIR RESULTADO
4 - SAIR
```

2.7. Código Fonte Completo

A seguir é apresentado o código-fonte completo do programa Sprint 1.

[Link do Github](#)